

## PRACTICAL NO 8

**Aim:** Write a program to implement Priority AVL Tree.

Balanced AVL Tree:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def get_height(self, node):
        if node is None:
            return 0
        return node.height

    def get_balance(self, node):
        if node is None:
            return 0
        return self.get_height(node.left) - self.get_height(node.right)

    def right_rotate(self, y):
        x = y.left
        temp = x.right
        x.right = y
        y.left = temp
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        temp = y.left
        y.left = x
        x.right = temp
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        return y
```

```

def insert(self, node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = self.insert(node.left, key)
    elif key > node.key:
        node.right = self.insert(node.right, key)
    else:
        return node
    node.height = 1 + max(self.get_height(node.left),
self.get_height(node.right))
    balance = self.get_balance(node)
    if balance > 1 and key < node.left.key:
        return self.right_rotate(node)
    if balance < -1 and key > node.right.key:
        return self.left_rotate(node)
    if balance > 1 and key > node.left.key:
        node.left = self.left_rotate(node.left)
        return self.right_rotate(node)
    if balance < -1 and key < node.right.key:
        node.right = self.right_rotate(node.right)
        return self.left_rotate(node)
    return node

# Inorder Traversal
def inorder(self, node):
    if node:
        self.inorder(node.left)
        print(node.key, end=" ")
        self.inorder(node.right)

avl = AVLTree()
root = None

values = [30, 20, 10, 25, 28, 40, 50]

for value in values:
    root = avl.insert(root, value)

print("Inorder traversal of AVL Tree:")
avl.inorder(root)

```

```

PS D:\Coding\college-pr> & C:\Pyth
Inorder traversal of AVL Tree:
10 20 25 28 30 40 50
PS D:\Coding\college-pr>

```

## Priority Queue:

```
class PriorityQueue:
    def __init__(self):
        self.queue = []

    def insert(self, value, priority):
        self.queue.append((priority, value))
        self.heapify_up(len(self.queue) - 1)

    def heapify_up(self, index):
        while index > 0:
            parent = (index - 1) // 2
            if self.queue[parent][0] >= self.queue[index][0]:
                break
            self.queue[parent], self.queue[index] = \
                self.queue[index], self.queue[parent]
            index = parent

    def remove(self):
        if len(self.queue) == 0:
            print("Priority Queue is empty")
            return None
        highest = self.queue[0]
        self.queue[0] = self.queue[-1]
        self.queue.pop()
        if self.queue:
            self.heapify_down(0)
        return highest

    def heapify_down(self, index):
        size = len(self.queue)
        while True:
            largest = index
            left = 2 * index + 1
            right = 2 * index + 2

            if left < size and self.queue[left][0] > self.queue[largest][0]:
                largest = left

            if right < size and self.queue[right][0] > self.queue[largest][0]:
                largest = right

            if largest == index:
                break
            self.queue[index], self.queue[largest] = \
                self.queue[largest], self.queue[index]
            index = largest
```

```

    def display(self):
        print("Priority Queue:")
        for priority, value in self.queue:
            print("Value:", value, "Priority:", priority)

pq = PriorityQueue()

pq.insert("Task A", 2)
pq.insert("Task B", 5)
pq.insert("Task C", 1)
pq.insert("Task D", 4)
pq.insert("Task E", 3)

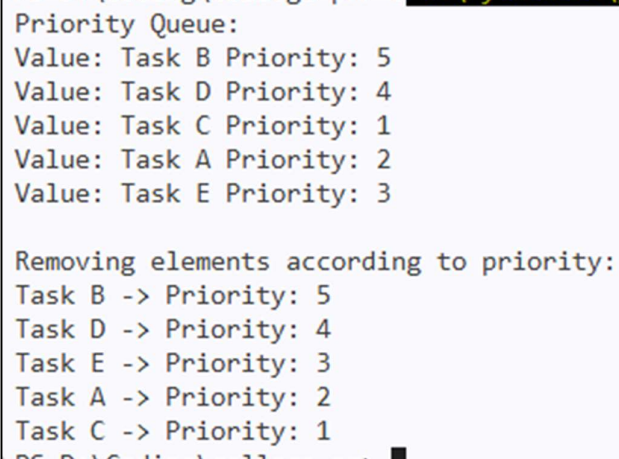
pq.display()

print("\nRemoving elements according to priority:")

while pq.queue:
    priority, value = pq.remove()
    print(value, "-> Priority:", priority)

```

OUTPUT:



```

Priority Queue:
Value: Task B Priority: 5
Value: Task D Priority: 4
Value: Task C Priority: 1
Value: Task A Priority: 2
Value: Task E Priority: 3

Removing elements according to priority:
Task B -> Priority: 5
Task D -> Priority: 4
Task E -> Priority: 3
Task A -> Priority: 2
Task C -> Priority: 1

```